HCCL算子开发入门（CCU引擎）



CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V5.0	2026/8/30	张兴平	全部	初版内容

目录

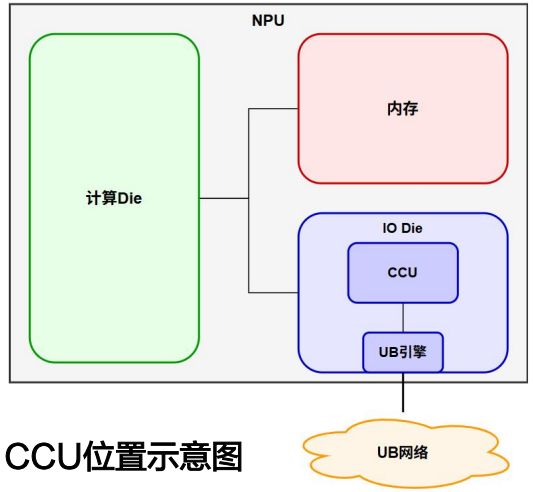
Part 1 CCU简介

Part 2 CCU编程模型和API介绍

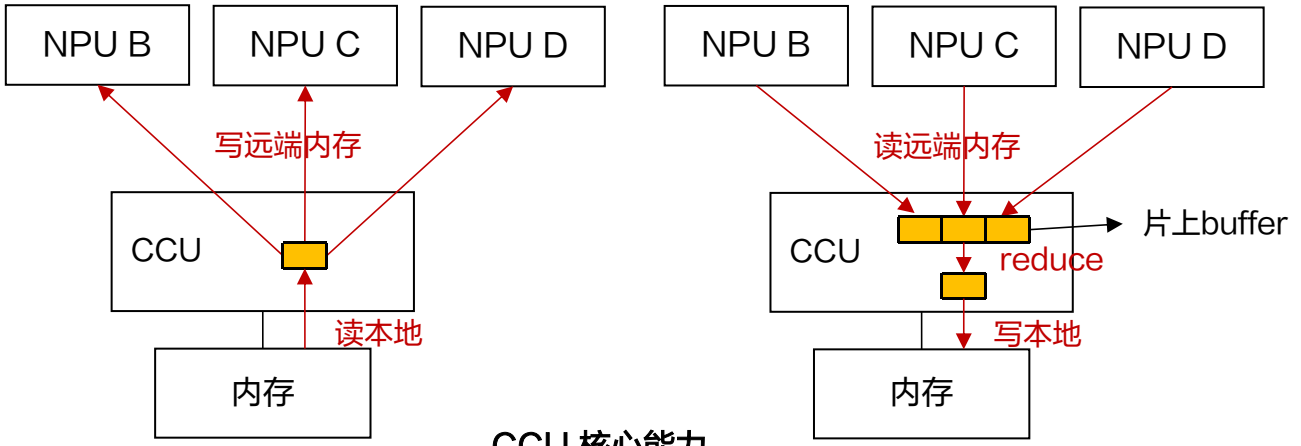
Part 3 HCCL算子开发-CCU引擎

CCU介绍

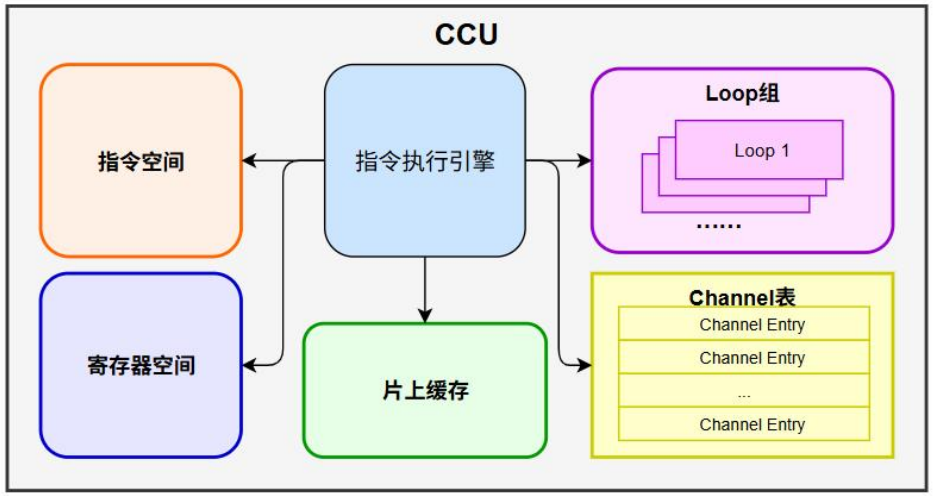
Collective Communication Unit（CCU）是Ascend 950上新增的用于加速集合通信的专用引擎，CCU根据软件预置的集合通信算法指令，完成通信节点间的同步、地址交换、数据传输以及规约计算，实现集合通信操作。



CCU位置示意图



CCU 核心能力



CCU 硬件逻辑架构

解决的问题及优势：

1、利用CCU片上buffer缓存通信数据，降低访存带宽需求；

2、利用片上buffer+内置计算单元执行Reduce操作，确保计算保序；

3、专用的通信调度与同步机制，降低时延，不占用计算算力；

4、消减用户buffer到CCL Buffer的拷贝

CANN

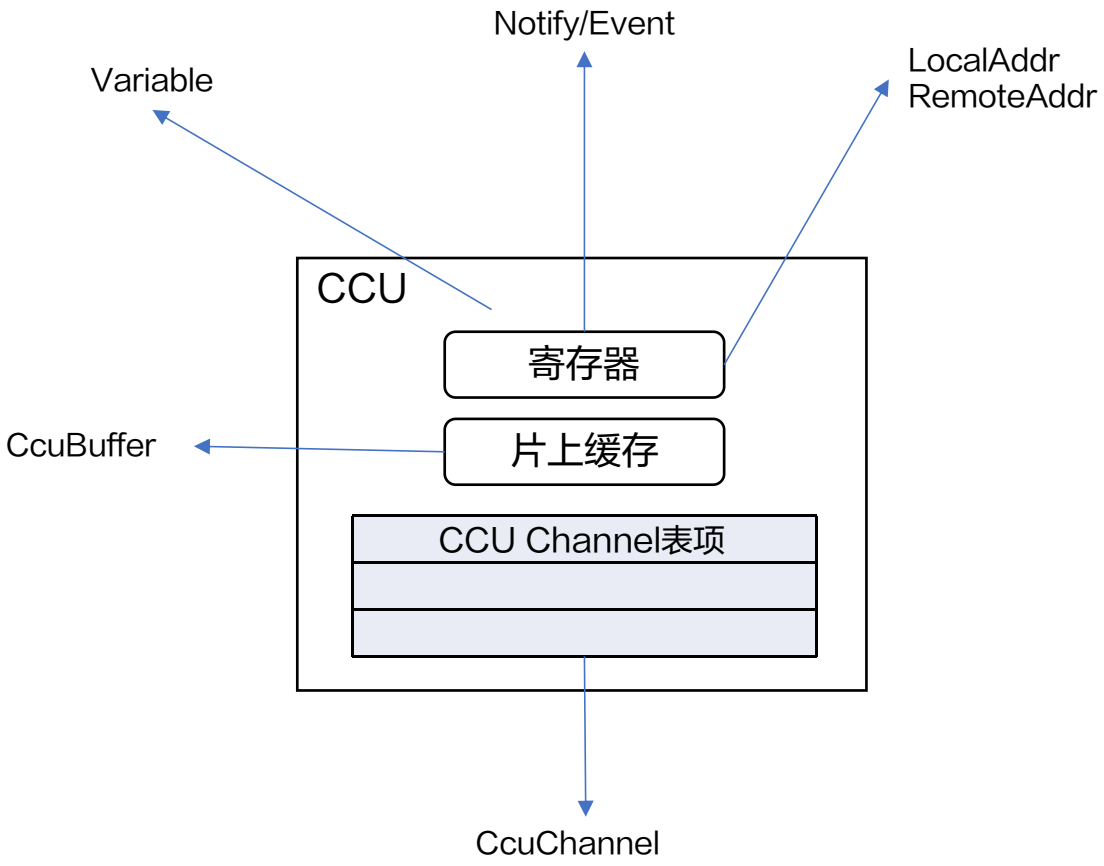
目录

Part 1 CCU简介

Part 2 CCU编程模型和API介绍

Part 3 HCCL算子开发-CCU引擎

CCU编程模型：资源抽象



资源类型	说明	对应资源
ccu::Variable	变量	通用寄存器
ccu::Address	地址	地址寄存器
ccu::Event	事件	同步寄存器
ccu::CcuBuffer	片上缓存分片，大小为4KB	CCU片上缓存
ccu::LocalAddr	本地地址，用于通信操作	包含地址和token，分别由Address和Variable保存
ccu::RemoteAddr	远端地址，用于通信操作	包含地址和token，分别由Address和Variable保存

资源创建：

- 单个资源创建：默认构造函数直接创建
- 批量连续资源创建：使用ccu::Array创建

注意：

1、Loop&LoopGroup对应的CCU并发操作对资源的连续性有要求；

2、使用原生数组创建资源（比如Variable vars[10]），不保证资源的连续性。

CCU编程模型：计算能力

通用寄存器&地址寄存器

计算能力：赋值和加法

使用立即数赋值

```
// 通用寄存器赋值
Variable x;
x = 5;

// 地址寄存器赋值
uint64_t ptr;
...
Address addr;
addr = ptr;
```

使用Variable赋值

```
Variable x, y;
...
Variable v;
v = x;

Address addr;
addr = y;
```

加法

```
Variable x=5, y=3, z;
z = x+y;

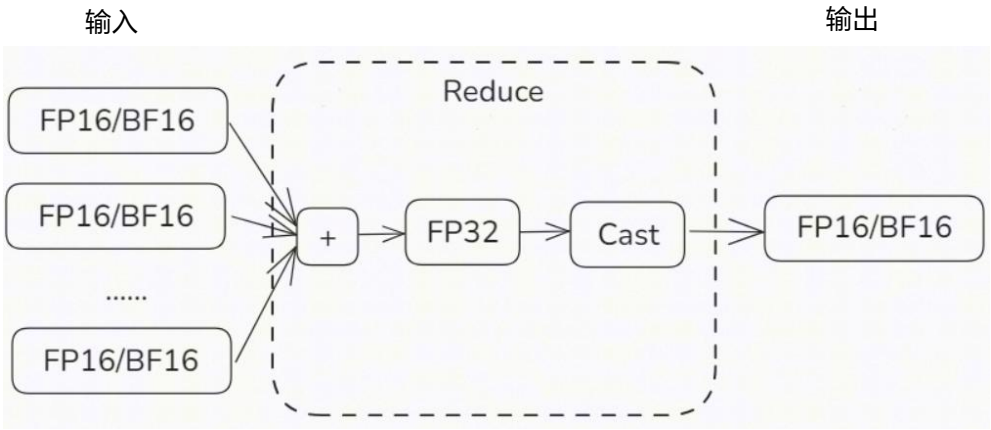
Address addr1, addr2;
...
addr1 += x;
addr2 = addr1 + x;
```

CcuBuffer

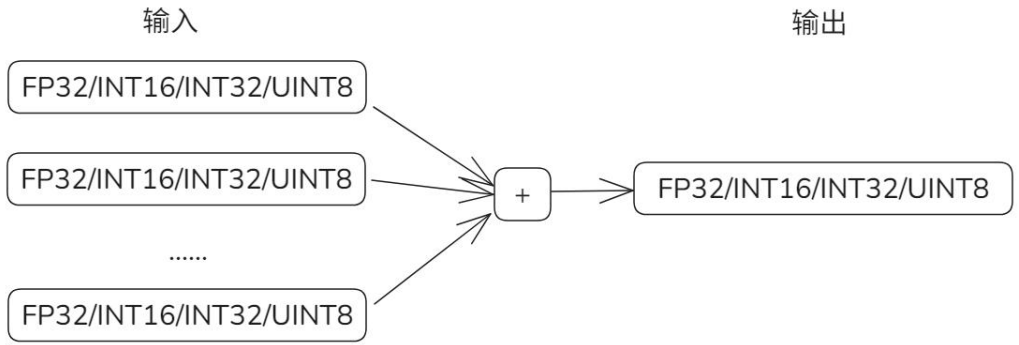
支持最多8个CcuBuffer的按序Reduce计算，支持的Reduce操作：ADD、MAX、MIN。

Reduce操作为ADD：

1）数据类型为FP16/BF16，内部会提升精度至FP32进行计算

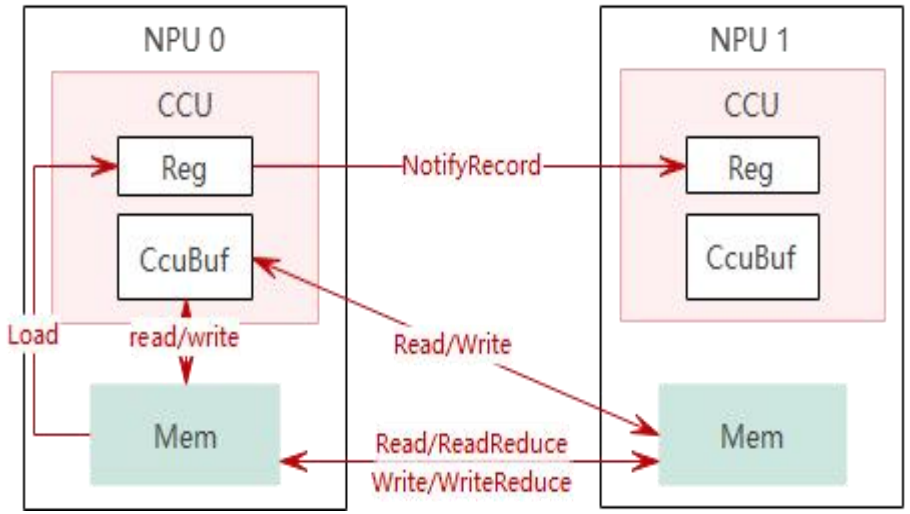


2）数据类型为FP32/INT16/INT32/UINT8



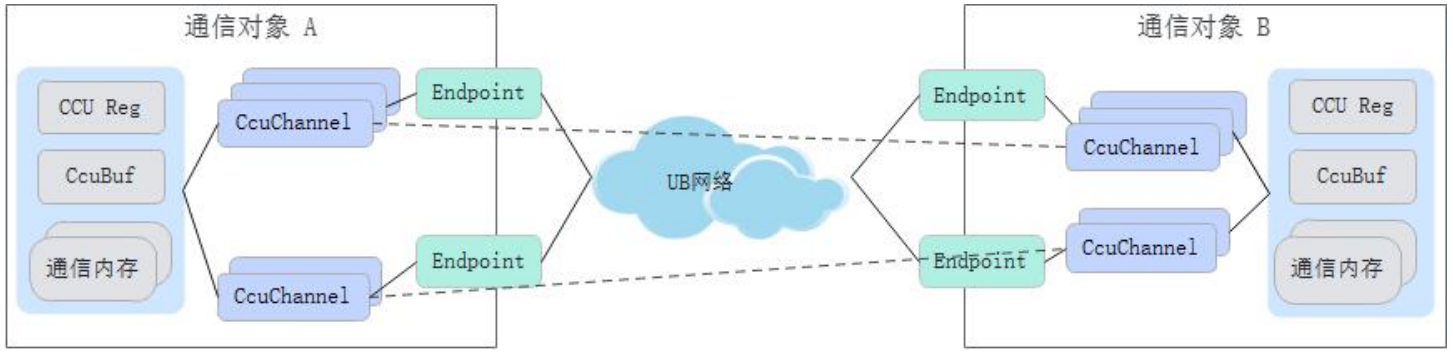
CCU编程模型：数据搬运 & 通信

CCU数据搬运能力

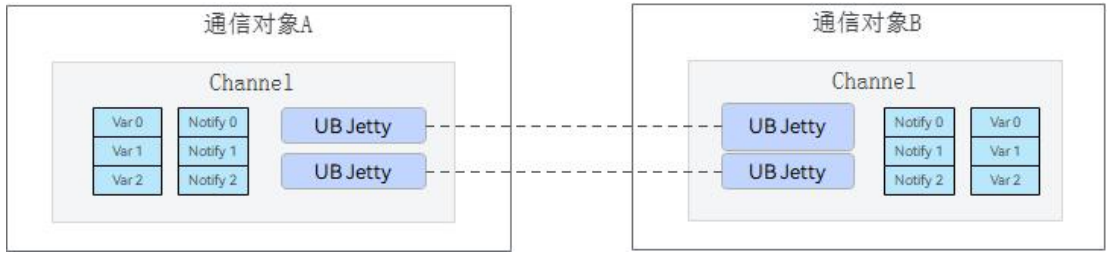


- Local Mem <-> Local Variable/Address Reg
- Local Mem <-> Local CcuBuf
- Local CcuBuf <-> Remote Mem
- Local Mem <-> Remote Mem

CCU通信模型



- Endpoint：通信的逻辑设备，包含通信协议与通信地址；
- CcuChannel：不同通信对象的Endpoint之间建立的通信通道；
- 开发者使用Channel可以读写远端通信对象内存或与远端通信对象进行同步。

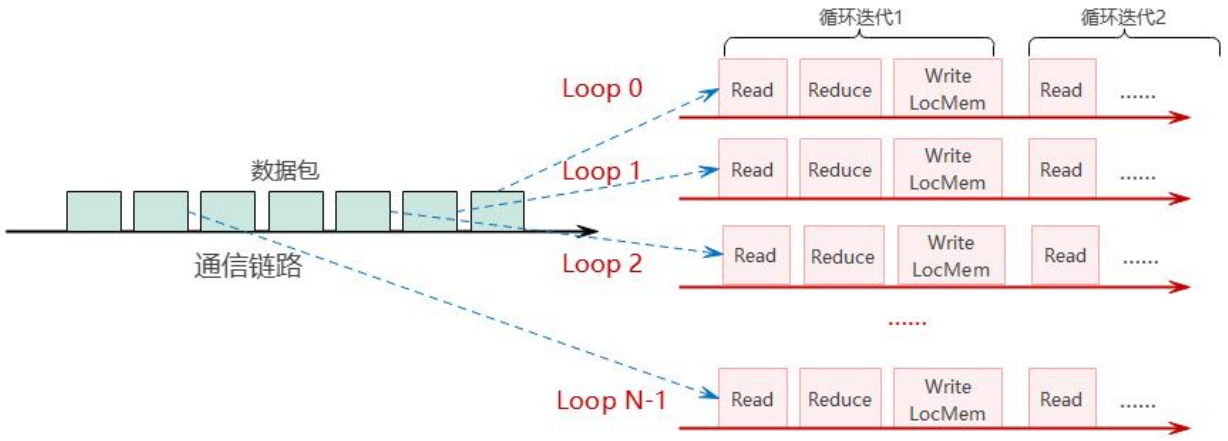


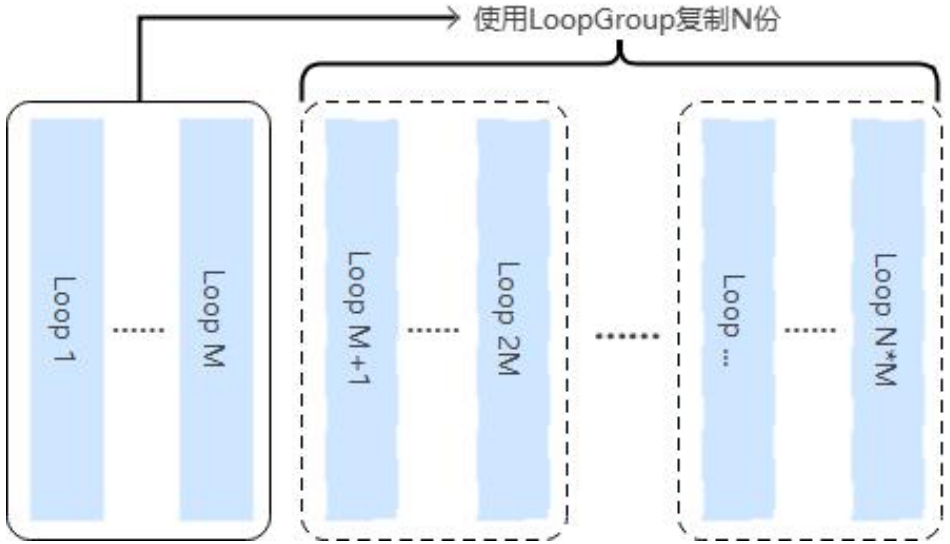
CcuChannel模型

CCU编程模型：并发模型

CCU并发模型

并发搬运数据，减少头开销，提升带宽利用率





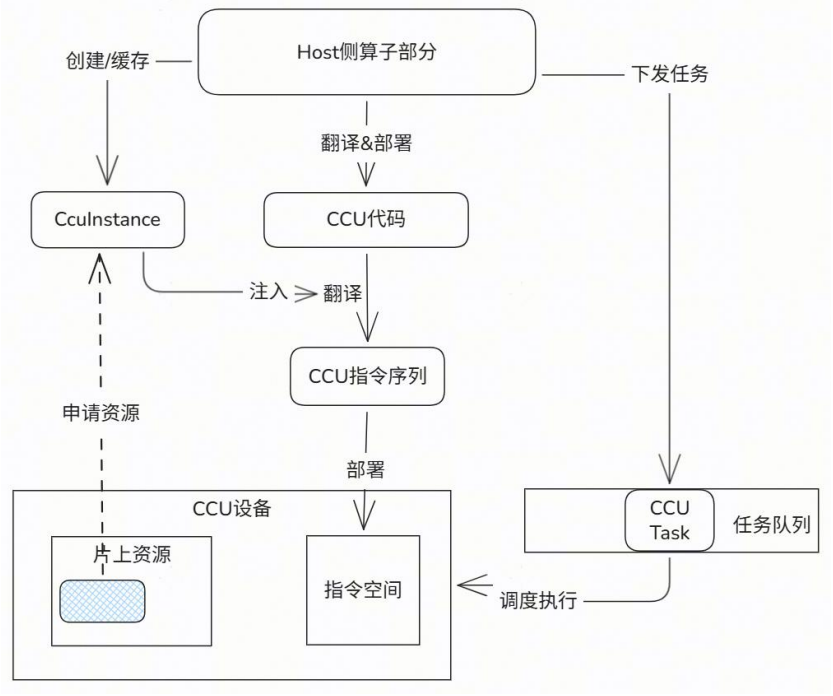
Loop：

- 一个CCU任务中的基础并发单元；
- Loop内的操作串行，Loop间并行；
- 配合CcuBuf循环搬运数据；
- Loop参数：循环次数、每次循环操作的内存地址增量偏移。

LoopGroup：

- 将一组Loop复制多份，增加并发Loop个数；
- 避免重复的指令逻辑手写多份。

CCU编程接口



CCU控制面接口

接口	说明
HcommCcuGetMemToken	基于内存地址获取CCU通信需要的token
HcclCommQueryCcuIns	查询通信域中已有的CCU实例，insNum<=1
HcommCcuKernelRegisterStart(ccuInsHandle)	与HcommCcuKernelRegisterEnd配对，Start和End之间可翻译多个可并行执行的CCU Kernel
HcommCcuKernelRegister(ccuInsHandle, dieId, kernelFuncName, kernelFunc, kernelArgs, argNum, *kernelHandle)	将CCU Kernel翻译为CCU指令并下发到设备中
HcommCcuKernelRegisterEnd(ccuInsHandle)	与HcommCcuKernelRegisterStart配对，在Start和End之间可翻译多个可并行执行的CCU Kernel

CCU数据面接口

分类		接口	说明
参数加载	-	ccu::LoadArg	加载第argIdx个参数到寄存器
		ccu::Load	从内存加载数据到寄存器
		ccu::Store	从寄存器保存数据到内存
数据搬运	本地	ccu::LocalCopy	LocalAddr↔LocalAddr、LocalAddr↔CcuBuffer、CcuBuffer↔LocalAddr
		ccu::LocalReduce	本地归约（CcuBuffer到Memory）、多CcuBuffer归约
	远端	ccu::Read	远端→本地（LocalAddr或CcuBuffer）
		ccu::Write	本地→远端（LocalAddr或CcuBuffer）
		ccu::ReadReduce	远端读+归约
		ccu::WriteReduce	本地写+归约
同步	本地	ccu::EventRecord	记录事件
		ccu::EventWait	等待事件
	远端	ccu::NotifyRecord	向远端发送通知
		ccu::NotifyWait	等待远端通知
		ccu::WriteVariableWithNotify	写变量并附带通知
流程控制	条件	ccu::CCU_IF	条件开始
		ccu::CCU_ELSE	条件分支
	循环	ccu::CCU_WHILE	while循环
	并发	ccu::Loop	执行引擎
		ccu::LoopGroup	循环组
	函数块	ccu::Func	微码函数块
		ccu::CallFunc	调用函数块

CANN

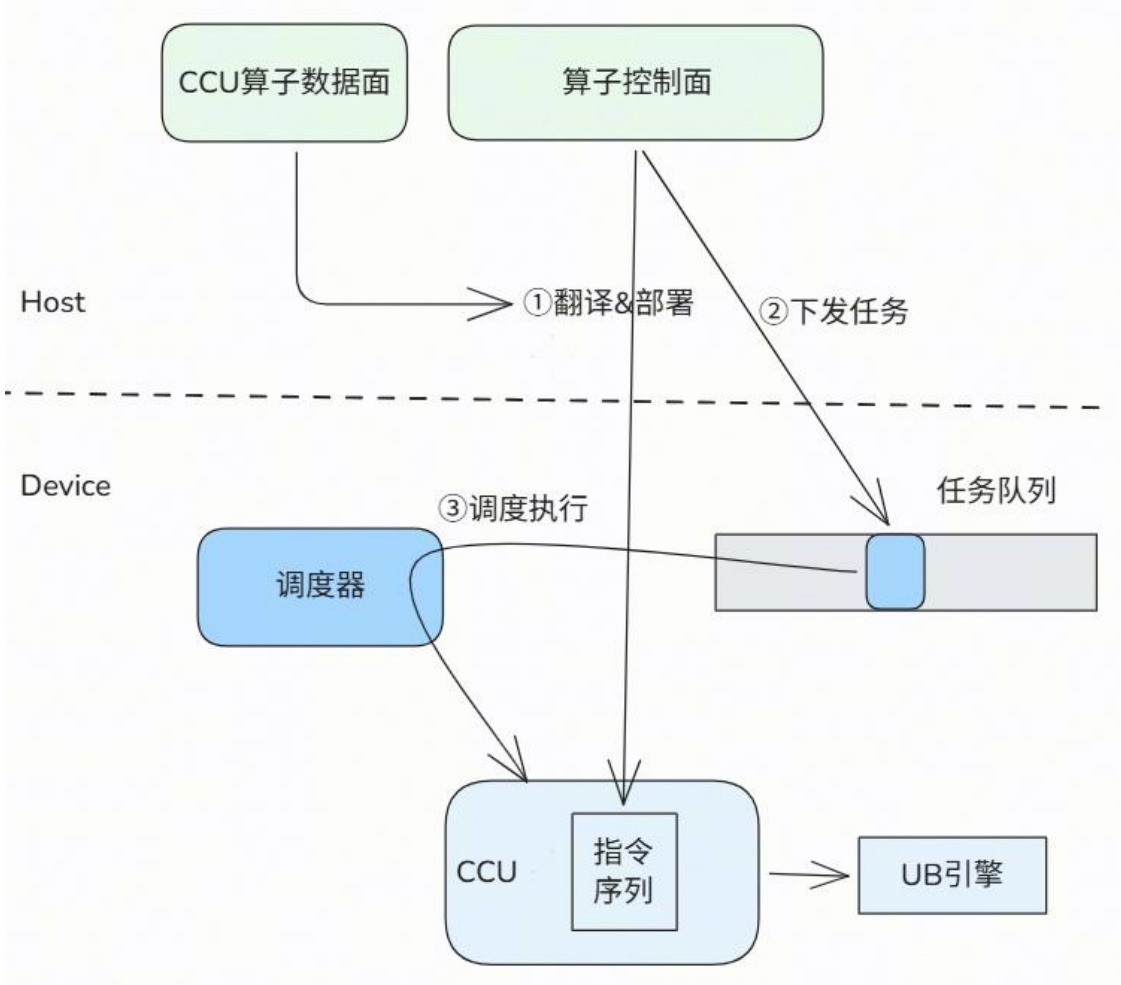
目录

Part 1 CCU简介

Part 2 CCU编程模型和API介绍

Part 3 HCCL算子开发-CCU引擎

HCCL算子执行流程：CCU通信引擎



算子控制面（算子入口）：

- 执行算法选择
- 申请&管理资源
- 翻译CCU算子数据面实现
- 下发部署翻译后的CCU指令流
- 下发CCU任务

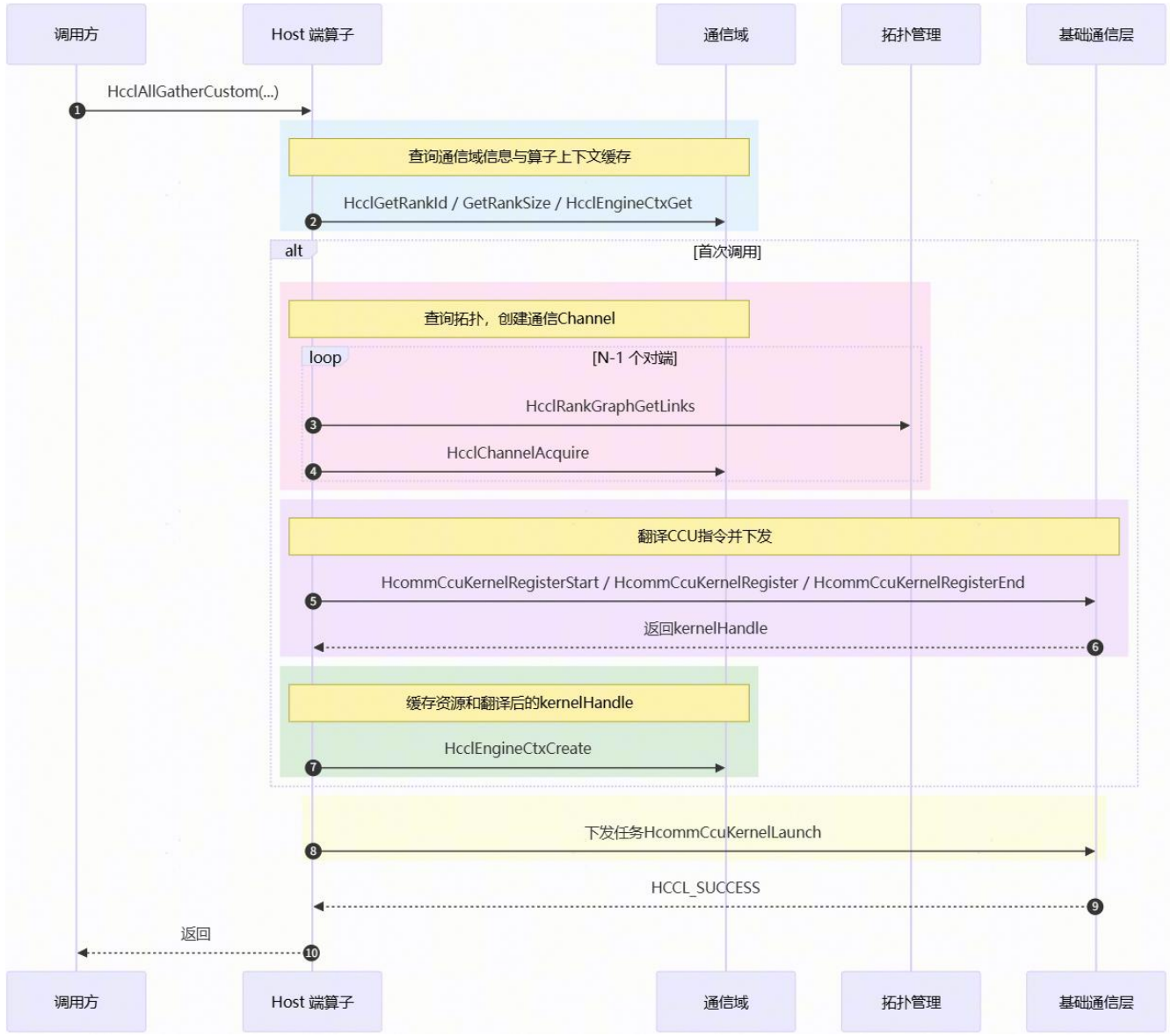
算子数据面（CCU Kernel）：

- 基于CCU数据面编程接口，实现通信算子的数据搬运逻辑
- 数据面程序需要翻译为CCU指令流并部署到CCU设备上
- Host下发到任务队列的任务被调度器调度至CCU执行
- CCU执行对应的指令流，并利用UB引擎完成数据搬运

CANN

HCCL算子开发：CCU AllGather算子样例

算子控制面开发




```
HcclResult HcclAllGatherCustom(void *sendBuf, void *recvBuf, uint64_t sendCount,
                               HcclDataType dataType, HcclComm comm, aclrtStream stream)
{
    // ========== 1. 获取通信域信息 ==========
    HcclGetRankId(comm, myRank);
    HcclGetRankSize(comm, rankSize);
    // ========== 2. 资源缓存检查 (Engine Ctx) ==========
    if (HcclEngineCtxGet(comm, tag, COMM_ENGINE_CCU, &ctx, &size) == SUCCESS) {
        resCtxHost.DeSerialize(ctxData);
    } else {
        // 2b. 申请 thread (封装主流, 创建通知)
        HcclThreadAcquireWithStream(comm, COMM_ENGINE_CCU, stream, notifyNum, &thread);
        resCtxHost.threads.push_back(thread);
        // 2c. 对每个远端 rank: 查询拓扑 + 申请 CCU 通道
        for (remoteRank = 0..rankSize) {
            if (remoteRank == myRank) continue;
            HcclRankGraphGetLinks(comm, layer, myRank, remoteRank, &linkList, &listSize);
            HcclChannelDescInit(&desc, 1);
            desc.remoteRank = remoteRank;
            ...
            HcclChannelAcquire(comm, COMM_ENGINE_CCU, &desc, 1, &channelHandle);
            kernelChannels.push_back(channelHandle);
        }
        // 2d. 查询 CCU 实例
        HcclCommQueryCcuIns(comm, &insHandle, &insNum);
        HcommCcuKernelRegisterStart(insHandle);
        HcommCcuKernelRegister(insHandle,
            "CcuAllGatherMesh1DMem2MemKernel",     // kernel 名称
            (void*)CcuAllGatherMesh1DMem2MemKernel, // kernel 函数指针
            kernelArg,                              // 含 channels 信息的参数
            &kernelHandle);                         // 输出 kernel 句柄
        HcommCcuKernelRegisterEnd(insHandle);
        resCtxHost.ccuKernels[0] = kernelHandle;
        // 2e. 序列化资源上下文, 写入 EngineCtx (下次复用)
        seq = resCtxHost.Serialize();
        HcclEngineCtxCreate(comm, tag, COMM_ENGINE_CCU, seq.size(), &ctx);
        memcpy(ctx, seq.data(), seq.size());
    }
    // ========== 3. 下发 CCU 任务 ==========
    // 获取内存 token (CCU 地址转换)
    HcommCcuGetMemToken(sendBuf, dataSize, &token);
    for (loop = 0; loop < loopCount; loop++) {
        // 计算分片地址 / 大小
        taskArgs = {inputAddr, outputAddr, token,
                    sliceInputOffset, sliceOutputOffset, sliceSize, ...};
        HcommCcuKernelLaunch(resCtx.threads[0], resCtx.ccuKernels[0],
                             taskArgs.data(), taskArgs.size());
    }
    return HCCL_SUCCESS;
}
```

参考链接：https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/ccu/op_host/allgather.cc

HCCL算子开发：CCU AllGather算子样例

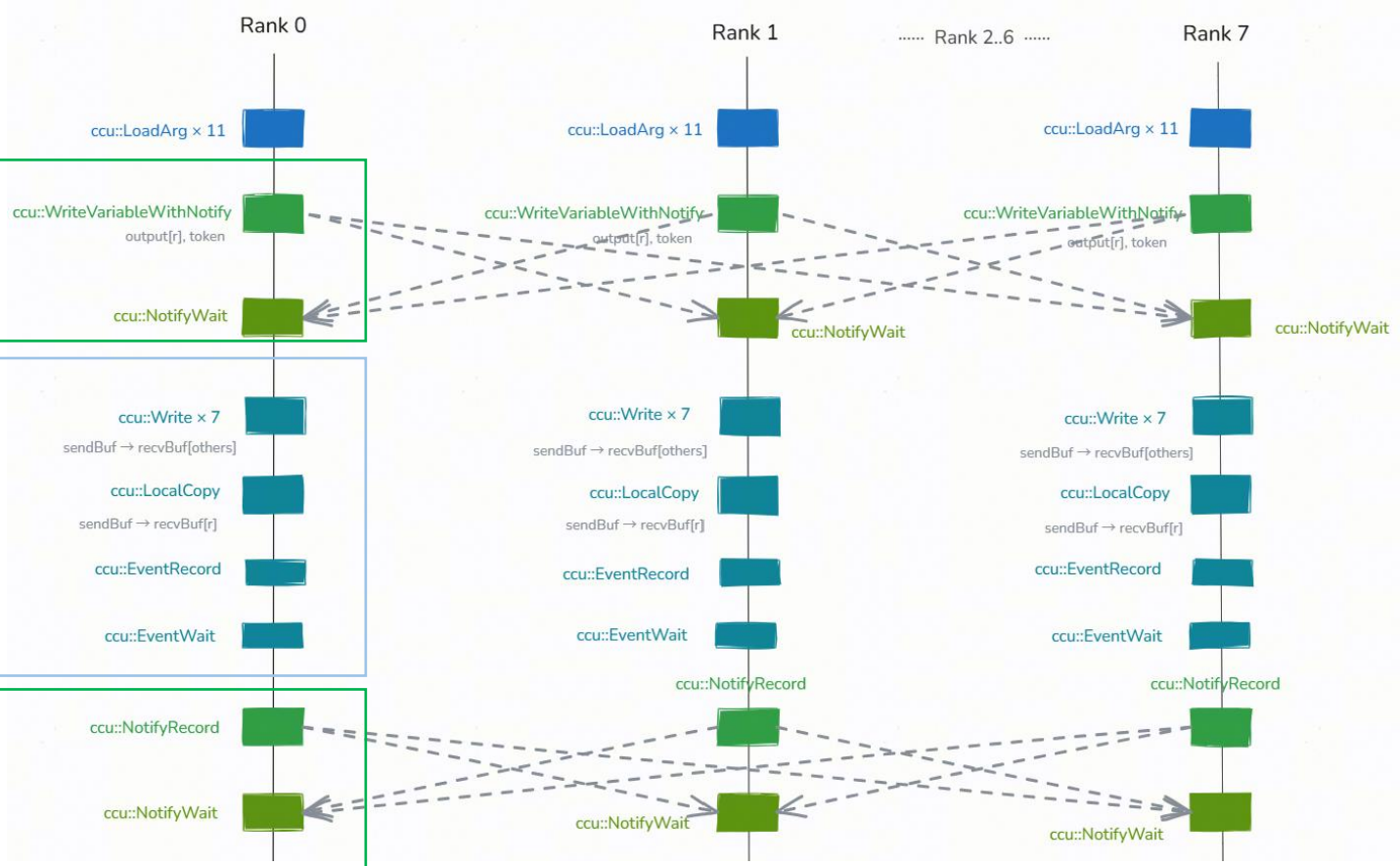
算子数据面（CCU Kernel）开发

```
CcuResult CcuAllGatherMesh1DMem2MemKernel(CcuKernelArg arg)
{
    // ... 省略 ① 资源初始化 / ② 加载参数

    // ③ 前同步: 用 WriteVariableWithNotify 一次性写变量+触发 notify
    for (auto ch : channels) {
        ccu::WriteVariableWithNotify(ch, myOutput, OUTPUT_XN_ID, CKE_IDX_0, ...);
        ccu::WriteVariableWithNotify(ch, myToken, TOKEN_XN_ID,  CKE_IDX_0, ...);
        ccu::NotifyWait(ch, CKE_IDX_0, OUTPUT_BIT | TOKEN_BIT);
    }

    // ④ 算法: 本地 + 跨卡并发, 共享 event 同步
    for (auto peer : ranks) {
        if (peer != myRank) {
            ccu::Write(channels[peer], remoteDst[peer], sendBuf,
                sliceSize, event, 1 << myRank);  // 跨卡写入
        } else {
            ccu::EventRecord(event, 1 << myRank);
            ccu::LocalCopy(...);  // 本卡: sendBuf -> recvBuf[myRank]
        }
    }
    ccu::EventWait(event, ALL_RANKS_MASK);  // 本卡任务全部发起

    // ⑤ 后同步: Record + Wait 等待对端完成
    for (auto ch : channels) {
        ccu::NotifyRecord(ch, CKE_IDX_0, POST_SYNC_BIT);
    }
    for (auto ch : channels) {
        ccu::NotifyWait(ch, CKE_IDX_0, POST_SYNC_BIT);
    }
    return CCU_SUCCESS;
}
```



参考链接：https://gitcode.com/cann/hccl/blob/master/examples/05_custom_ops_allgather/ccu/op_kernel_ccu/ccu_kernel.cc

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN